

Relatório de Auditoria de Segurança

— Guelfiness —

Data: 28/08/2026

Escopo auditado: repositório completo (9 arquivos versionados + histórico git): `index.html`, `assets/js/main.js`, `assets/css/style.css`, `Dockerfile`, `docker-compose.yml`, `docker-compose.local.yml`, `.github/workflows/oci-deploy.yml`, `.gitignore`, `.dockerignore` e todos os commits do histórico.

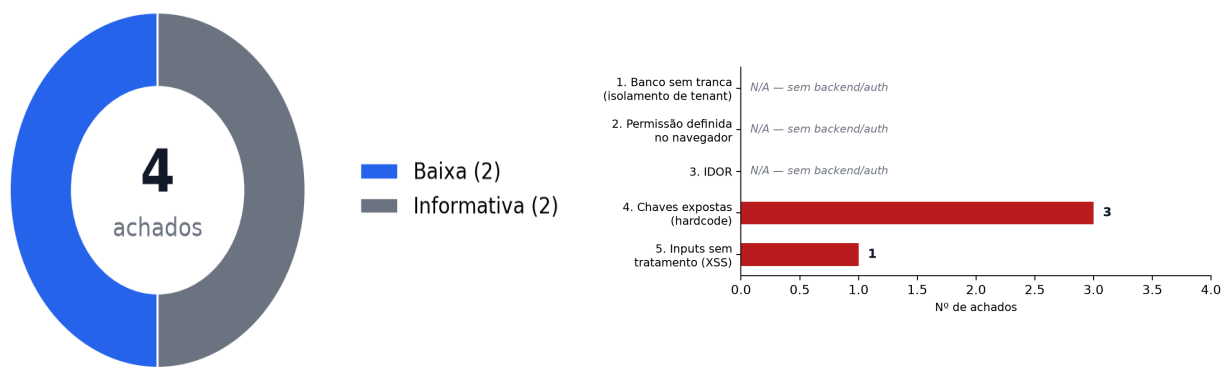
Nota metodológica: a stack detectada é **site estático (HTML/CSS/JS vanilla) servido por nginx:alpine em Docker, com deploy via GitHub Actions → SSH/rsync → VM OCI**. Não há backend, banco de dados, ORM, autenticação nem multi-tenancy. As cinco categorias foram mapeadas para essa stack: (1) isolamento de tenant, (2) autorização no servidor e (3) IDOR foram verificadas como **não aplicáveis** — não existe superfície de servidor no projeto; (4) chaves expostas foi aplicada a código, configs, Docker, CI e histórico git; (5) inputs/XSS foi aplicada ao JavaScript do frontend, ao HTML estático e às defesas da camada de entrega (headers HTTP).

1. Resumo executivo

A auditoria cobriu 100% do código-fonte e do histórico git do projeto. Foram registrados **4 achados**: **0 críticos**, **0 altos**, **0 médios**, **2 baixos** e **2 informativos**, além de **10 pontos fortes** verificados. Nenhum dos achados é explorável diretamente hoje: três dependem de condições futuras de processo e um é de defesa em profundidade. A postura geral de segurança do projeto é **boa**, coerente com um site estático sem superfície de servidor.



Achados por severidade e por categoria



Categorias 1–3 constam como *N/A*: sem backend, autenticação ou banco de dados, não há queries, rotas privilegiadas ou handlers por ID para auditar — verificado por inspeção completa da árvore e do histórico.

2. Pontos fortes e pontos fracos

2.1 Pontos fortes (verificados, com evidência)

✓ Categorias 1, 2 e 3 não aplicáveis — verificado

O repositório contém apenas 9 arquivos versionados (site estático + infra). Não há backend, banco de dados, ORM, autenticação, papéis ou handlers de rota: isolamento de tenant, verificação de privilégio no servidor e IDOR não têm superfície neste projeto. Verificado por inspeção completa da árvore e do histórico git.

✓ Segredos somente via GitHub Secrets, com validação de startup

.github/workflows/oci-deploy.yml:16-35 valida a presença de SSH_HOST, SSH_USER e SSH_PRIVATE_KEY no início do job e aborta com ::error:: explícito se algum faltar. Nenhum valor de segredo aparece no código — apenas referências \${ secrets.* }.

✓ Histórico git limpo de segredos

Busca por padrões de segredo (api_key, secret, password, token, BEGIN PRIVATE KEY, AKIA, ghp_, sk-, xox*, bearer) em TODOS os commits retornou apenas as referências \${ secrets.* } do workflow e a palavra 'JWT' em texto descritivo do portfólio. Nenhum segredo foi commitado em nenhuma revisão.

✓ Imagem Docker não vaza metadados do repositório

.dockerignore:1-9 exclui .git, .github, Dockerfile, docker-compose*.yml e o próprio .dockerignore do contexto copiado para a imagem.

✓ JS sem sumidouros de XSS; texto dinâmico sempre via textContent

assets/js/main.js não usa eval, new Function nem document.write. Todo texto dinâmico é atribuído via textContent (linhas 259, 313, 335, 353, 367). O único uso de HTML bruto — insertAdjacentHTML em main.js:53 — consome projectTrack.innerHTML, markup estático da própria página (auto-clonagem do carrossel), sem entrada de usuário.

✓ Script inline de tema não executa dado do localStorage

index.html:12-18 lê guelfiness-theme do localStorage e aplica via setAttribute ('data-theme', ...): contexto de atributo, sem execução de código. Não explorável.

✓ Links externos protegidos contra tabnabbing

Todos os ~20 links com target="_blank" do index.html carregam rel="noopener".

✓ Higiene de SSH no pipeline de deploy

A chave privada é gravada com chmod 600 (oci-deploy.yml:42-43), o host é registrado via ssh-keyscan -H em known_hosts (:46-51) e o rsync exclui .git e .github do upload (:61-64).

✓ Superfície de rede mínima nos containers

docker-compose.yml (produção) não publica portas — o container só é alcançável pela rede do reverse proxy. docker-compose.local.yml monta os arquivos como read-only (:ro) e define healthcheck.

✓ Sem dependências de runtime de terceiros no frontend

O site é HTML/CSS/JS puro, sem node_modules nem CDNs de JavaScript — a única origem externa é Google Fonts. Supply chain de JS praticamente inexistente.

2.2 Pontos fracos (riscos centrais)

Nenhum risco crítico ou alto foi identificado. Os riscos centrais concentram-se na **cadeia de build e deploy**: (a) o workflow tolera credenciais SSH em variables (F1); (b) o Dockerfile confia cegamente no .dockerignore para não vaziar arquivos do repositório (F3) e não pinha a imagem base (F2); (c) a camada de entrega não envia security headers (F4). São riscos de processo e de defesa em profundidade — todos com correção simples, detalhada na seção 4.

3. Achados detalhados

F1 — Fallback para Actions Variables permite chave privada SSH fora de Secrets

BAIXA

4. Chaves expostas

.github/workflows/oci-deploy.yml:21 e :39

Descrição: O workflow aceita SSH_PRIVATE_KEY (e SSH_HOST/SSH_USER) vindos de variáveis de repositório (vars) como fallback de secrets: `${{ secrets.SSH_PRIVATE_KEY || vars.SSH_PRIVATE_KEY }}`. Variables do GitHub Actions não são mascaradas nos logs e ficam visíveis em texto claro para todo colaborador com acesso de leitura ao repositório. O fallback incentiva cadastrar a chave privada — que dá acesso de deploy ao servidor de produção (OCI) — no mecanismo errado.

Explorabilidade: Condicional: só é explorável se o operador cadastrar a chave como variable. Nesse cenário, qualquer colaborador com read no repo lê a chave privada e pode acessar o servidor de produção via SSH.

F2 — Imagem base nginx:alpine sem pin de versão ou digest

INFORMATIVA

4. Chaves expostas

Dockerfile:1

Descrição: FROM nginx:alpine usa tag móvel: cada build pode baixar uma imagem diferente, sem revisão. Builds não são reproduzíveis e, num cenário de supply chain (tag republicada com conteúdo malicioso), não há digest para detectar a mudança.

Explorabilidade: Condicional: requer comprometimento upstream da imagem ou erro de atualização. Sem exploração direta a partir do código do projeto.

F3 — COPY . joga o repositório inteiro no docroot público do nginx

INFORMATIVA

4. Chaves expostas

Dockerfile:3 (com .dockerignore:1-9)

Descrição: COPY . /usr/share/nginx/html copia todo o contexto de build para o diretório servido publicamente. A única barreira contra vazamento é o .dockerignore — hoje correto (exclui .git, .github, Dockerfile, composes). Qualquer arquivo sensível adicionado no futuro (.env, backup, anotação com credencial) sem atualizar o .dockerignore vira conteúdo público, servido pelo nginx, sem nenhum erro de build.

Explorabilidade: Condicional: depende de erro futuro de processo. Hoje o .dockerignore cobre os arquivos sensíveis existentes.

F4 — Ausência de security headers HTTP (CSP, X-Frame-Options e correlatos)

BAIXA

5. Inputs sem tratamento (XSS)

Dockerfile:1-7 / index.html:1-19 (ausência de configuração)

Descrição: O projeto não tem nginx.conf customizado: o container usa a configuração default da imagem nginx:alpine, que não envia Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy nem Permissions-Policy. O index.html também não define meta CSP. Não há defesa em profundidade contra XSS (caso algum dia o site passe a processar entrada de usuário) nem contra clickjacking.

Explorabilidade: Risco prático baixo hoje: o site é 100% estático e não processa entrada de usuário. O achado é de defesa em profundidade.

3.1 Tabela consolidada

Sev.	Arquivo:linha	Descrição
BAIXA	.github/workflows/oci-deploy.yml :21 e :39	F1 — Fallback para Actions Variables permite chave privada SSH fora de Secrets
INFORMATIVA	Dockerfile:1	F2 — Imagem base nginx:alpine sem pin de versão ou digest
INFORMATIVA	Dockerfile:3 (com .dockerignore:1-9)	F3 — COPY . joga o repositório inteiro no docroot público do nginx
BAIXA	Dockerfile:1-7 / index.html:1-19 (ausência de configuração)	F4 — Ausência de security headers HTTP (CSP, X-Frame-Options e correlatos)

4. Recomendações priorizadas

P1 Remover o fallback para vars no workflow de deploy (F1)

Trocar `${{ secrets.X || vars.X }}` por `${{ secrets.X }}` para `SSH_HOST`, `SSH_USER` e `SSH_PRIVATE_KEY` em `.github/workflows/oci-deploy.yml`, e documentar no README que a chave privada nunca deve ser cadastrada como variable.

P2 Adicionar `nginx.conf` com security headers e pinar a imagem base (F4, F2)

Criar um `nginx.conf` próprio enviando `Content-Security-Policy`, `X-Frame-Options` (ou `frame-ancestors`), `X-Content-Type-Options: nosniff`, `Referrer-Policy` e `Permissions-Policy`; no Dockerfile, pinar a imagem por digest (`nginx:alpine@sha256:...`).

P3 Estreitar o COPY do Dockerfile (F3)

Substituir `COPY .` por cópias explícitas de `index.html` e `assets/` — assim nenhum arquivo futuro do repositório entra na imagem por acidente, independente do `.dockerignore`.

P4 Pinar actions por SHA e avaliar HSTS no reverse proxy

Usar `actions/checkout@<sha-do-commit>` no workflow (supply chain do CI) e, no Nginx central da OCI, habilitar `Strict-Transport-Security` para `guelfi.com.br`.

5. Issues para o GitHub

Cada bloco abaixo contém o texto completo de uma issue em Markdown, pronto para copiar e colar. Achados triviais relacionados foram agrupados em issue única (F2+F3 → Issue 2) para evitar spam.

Issue 1: [Segurança] Fallback para vars permite chave privada SSH fora de Secrets no deploy

```

--- ISSUE 1 ---
**Título:** [Segurança] Fallback para vars permite chave privada SSH fora de Secrets no deploy
**Labels:** security, severidade-baixa

## Problema

O workflow de deploy aceita as credenciais SSH tanto de secrets quanto de variables do repositório, com `secrets` como primeira opção e `vars` como fallback:

```yaml
SSH_PRIVATE_KEY: ${ secrets.SSH_PRIVATE_KEY || vars.SSH_PRIVATE_KEY }
```

Variables do GitHub Actions não são mascaradas nos logs e ficam visíveis em texto claro para qualquer colaborador com acesso de leitura ao repositório. O fallback incentiva cadastrar a chave privada SSH – que dá acesso de deploy ao servidor de produção (OCI) – no mecanismo errado.

## Por que é explorável

Se a chave for cadastrada como variable (o fallback sugere que isso é válido), qualquer pessoa com read no repo lê a chave privada e pode abrir sessão SSH no servidor de produção. Condição: o operador precisa usar o fallback – hoje os valores aparentemente estão em secrets, mas nada impede o uso inseguro.

## Evidência

`.github/workflows/oci-deploy.yml:19-21` e `:38-39`:

```yaml
SSH_HOST: ${ secrets.SSH_HOST || vars.SSH_HOST }
SSH_USER: ${ secrets.SSH_USER || vars.SSH_USER }
SSH_PRIVATE_KEY: ${ secrets.SSH_PRIVATE_KEY || vars.SSH_PRIVATE_KEY }
```

## Impacto

Comprometimento da chave privada de deploy → acesso SSH ao servidor OCI que hospeda todos os produtos (o mesmo host serve vários SaaS, conforme a seção Infraestrutura do próprio site).

## Sugestão de correção

Remover o `|| vars.*` de todas as credenciais sensíveis, mantendo apenas `${ secrets.* }`, e documentar no README que `SSH_PRIVATE_KEY` jamais deve ser cadastrada como variable.

## Critérios de aceite

- [ ] Nenhuma ocorrência de `vars.SSH_PRIVATE_KEY`, `vars.SSH_USER` ou `vars.SSH_HOST` no workflow
- [ ] Deploy continua funcionando com os valores em secrets
- [ ] README (ou comentário no workflow) registra a política: credenciais somente em Secrets
--- FIM ISSUE 1 ---

```

Issue 2: [Segurança] Dockerfile: imagem nginx sem pin e COPY . amplo dependente de .dockerignore

```

--- ISSUE 2 ---
**Título:** [Segurança] Dockerfile: imagem nginx sem pin e COPY . amplo dependente de .dockerignore
**Labels:** security, severidade-informativa

## Problema

Dois pontos de endurecimento no Dockerfile (agrupados por serem do mesmo tema –
cadeia de build):

1. `FROM nginx:alpine` usa tag móvel, sem pin de versão ou digest. Builds não
   são reproduzíveis e não há como detectar uma imagem base alterada upstream.
2. `COPY . /usr/share/nginx/html` copia o repositório inteiro para o docroot
   público do nginx. A única proteção contra vazar arquivos sensíveis é o
   `.dockerignore` – hoje correto, mas qualquer arquivo sensível futuro (`.env`,
   backup, anotação) adicionado sem atualizar o `.dockerignore` será servido
   publicamente, sem erro de build.

## Por que é explorável

Não há exploração direta hoje. O risco é de processo: (1) um rebuild silencioso
pode puxar uma imagem base diferente/comprometida; (2) um único arquivo
sensível commitado no futuro vaza publicamente pela porta 80 do container.

## Evidência

`Dockerfile:1-3`:

```dockerfile
FROM nginx:alpine

COPY . /usr/share/nginx/html
```

`.dockerignore:1-9` (proteção atual, baseada em lista de exclusão):

```
.
.git
.github
docker-compose.yml
docker-compose.local.yml
Dockerfile
.dockerignore
README.md
```

## Impacto

Vazamento futuro de arquivos sensíveis do repositório via HTTP; ingestão não
revisada de imagem base em cenário de supply chain.

## Sugestão de correção

- Pinar a imagem base por digest: `FROM nginx:alpine@sha256:<digest>`
- Substituir `COPY .` por cópias explícitas:

```dockerfile
COPY index.html /usr/share/nginx/html/index.html
COPY assets/ /usr/share/nginx/html/assets/
```

## Critérios de aceite

- [ ] Imagem base pinada por digest no Dockerfile
- [ ] Imagem final contém apenas `index.html` e `assets/` no docroot
  (verificar com `docker run --rm <img> ls -R /usr/share/nginx/html`)
- [ ] Site continua servindo corretamente após o rebuild
--- FIM ISSUE 2 ---

```

Issue 3: [Segurança] Ausência de security headers HTTP (CSP, X-Frame-Options e correlatos)


```

--- ISSUE 3 ---
**Título:** [Segurança] Ausência de security headers HTTP (CSP, X-Frame-Options e correlatos)
**Labels:** security, severidade-baixa

## Problema

O projeto não possui `nginx.conf` customizado: o container usa a configuração
default da imagem `nginx:alpine`, que não envia nenhum security header. O
`index.html` também não define `<meta>` de CSP. Headers ausentes:

- `Content-Security-Policy`
- `X-Frame-Options` (ou `frame-ancestors` na CSP)
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy`
- `Permissions-Policy`

## Por que é explorável

O risco prático é baixo hoje — o site é 100% estático e não processa entrada de
usuário. O achado é de **defesa em profundidade**: sem CSP, qualquer futura
funcionalidade que injete conteúdo (ou um endpoint de terceiro comprometido)
executa scripts sem restrição; sem X-Frame-Options, a página pode ser emoldurada
para clickjacking.

## Evidência

`Dockerfile:1-7` — não copia nenhum `nginx.conf`; a imagem default não envia os
headers. `index.html:1-19` — `<head>` sem meta CSP.

Verificável em produção com:

```bash
curl -sI https://www.guelfi.com.br/ | grep -iE 'content-security|x-frame|x-content|referrer-policy'
```

(sem resultados hoje)

## Impacto

Ausência de mitigação em camadas contra XSS e clickjacking; penalização em
scanners de segurança (securityheaders.com, observatory).

## Sugestão de correção

Adicionar um `nginx.conf` ao repositório e copiá-lo no Dockerfile:

```nginx
server {
 listen 80;
 root /usr/share/nginx/html;
 add_header Content-Security-Policy "default-src 'self'; style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com; font-src https://fonts.gstatic.com; img-src 'self' data;
script-src 'self' 'unsafe-inline' always;
 add_header X-Frame-Options "DENY" always;
 add_header X-Content-Type-Options "nosniff" always;
 add_header Referrer-Policy "strict-origin-when-cross-origin" always;
 add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
}
```

(`'unsafe-inline'` é necessário para o script de tema e estilos inline
existentes; pode ser eliminado depois movendo-os para arquivos.)

## Critérios de aceite

- [ ] `curl -sI` no site em produção mostra os 5 headers
- [ ] Site renderiza sem erros de CSP no console do navegador
- [ ] Nota A ou superior em securityheaders.com
--- FIM ISSUE 3 ---

```